

MINI ANTIVIRUS SCANNER

Prepared By: Minahil Tariq

1. Introduction

In today's rapidly evolving digital landscape, cybersecurity threats such as viruses, trojans, ransomware, and hacking tools pose a serious risk to personal and organizational data. Traditional antivirus solutions are built on high-level frameworks and rely on operating system APIs for their functionality. This project takes a different, more fundamental approach.

The Mini Antivirus Scanner is a complete two-layer cybersecurity application consisting of a low-level core written in x86 Assembly Language and a modern graphical user interface developed in Python using the Tkinter framework. Together, the two components demonstrate both the foundational mechanics of threat detection at the machine level and the usability improvements that a well-designed GUI brings to a security tool.

The Assembly backend operates as a standalone console application capable of scanning filenames against a keyword-based threat dictionary, maintaining a persistent quarantine list, and generating colour-coded scan reports. The Python GUI frontend extends this concept into a fully interactive desktop application that can scan real files on disk reading their actual content and display results in a polished, dark-themed interface.

2. Objectives

- Develop a console-based antivirus scanner in x86 Assembly Language demonstrating low-level programming skills
 - Implement a secure login system with username/password authentication in the Assembly application
 - Provide multiple scan modes in Assembly: Quick Scan, Full Scan, and Custom Scan
 - Design and implement a Python GUI frontend that performs real file scanning by reading actual file content
 - Detect threats using both filename-based keyword matching and content-based string inspection
 - Maintain a persistent quarantine list tracking all detected threats across scan sessions
 - Generate formatted scan reports with threat level classification (LOW / MEDIUM / HIGH)
 - Apply and demonstrate core Assembly concepts: string comparison, stack discipline, procedure calls, and memory management
-

- Apply Python GUI design principles using Tkinter to create an intuitive, professional security tool interface

3. Project Overview Two-Layer Architecture

This project is composed of two independent but complementary components:

Layer	Component	Description
Layer 1 Core	Assembly Console App (.asm)	Demonstrates low-level threat detection logic, login system, scan modes, and quarantine using x86 MASM
Layer 2 GUI	Python Tkinter GUI App (.py)	A fully interactive desktop antivirus scanner with a dark-themed GUI that scans real files on disk using filename, extension, and content analysis

Both layers implement the same core concept identifying threats using keyword/pattern matching but operate independently. The Assembly component shows mastery of low-level COAL concepts; the Python GUI component demonstrates how the same security logic can be delivered as a practical, user-friendly product.

4. Tools and Technologies Used

Tool	Used In	Purpose
x86 Assembly (MASM)	Assembly Layer	Core backend language; all scan logic, login, and quarantine
Irvine32 Library	Assembly Layer	Console I/O: WriteString, ReadString, ReadInt, SetTextColor
Visual Studio Community	Assembly Layer	IDE for writing, assembling, linking, and debugging .asm
Windows Console (CMD)	Assembly Layer	Execution environment for the compiled assembly program
Python 3.x	GUI Layer	Primary language for the frontend application

Tkinter (tk / ttk)	GUI Layer	Standard Python GUI toolkit for windows, frames, and widgets
filedialog (tkinter)	GUI Layer	Opens OS file-picker dialogs for single file, multi-file, and folder selection

5. System Design

5.1 Assembly Backend Data Layout

- Title & UI Strings: Null-terminated BYTE arrays for banners, menus, and prompts
- Login Credentials: Hardcoded correct username (minahil) and password (1234) with 30byte input buffers
- File Database: 8 built-in filenames (50 bytes each) + 5 extra user-added slots
- Keyword Dictionary: Four threat keywords stored as strings — virus, hack, malware, trojan
- Quarantine Array: 20 slots of 50 bytes each with a persistent quarantineCount (never reset)
- Per-Scan Counters: filesScanned, threatCount, safeCount — reset at the start of each new scan

5.2 Assembly Backend Procedure Map

Procedure	Responsibility
main	Entry point; shows banner, calls LoginSystem, runs the main menu loop
LoginSystem	Validates username and password; loops on failure with retry message
DetectThreat	Core engine; checks filename against keyword dictionary, quarantines threats, shows threat level
ContainsWord	Substring search (haystack/needle); returns EAX=1 if keyword is found anywhere in filename
MyStrCompare	Exact null-terminated string comparison used by the login system
MyCopy	Byte-by-byte string copy; used to store threat filenames into quarantine slots
ShowReport	Prints colour-coded scan summary: files scanned, threats, safe files
ResetCounters	Zeroes filesScanned / threatCount / safeCount before each scan; quarantineCount is NOT reset

5.3 Python GUI Application Flow

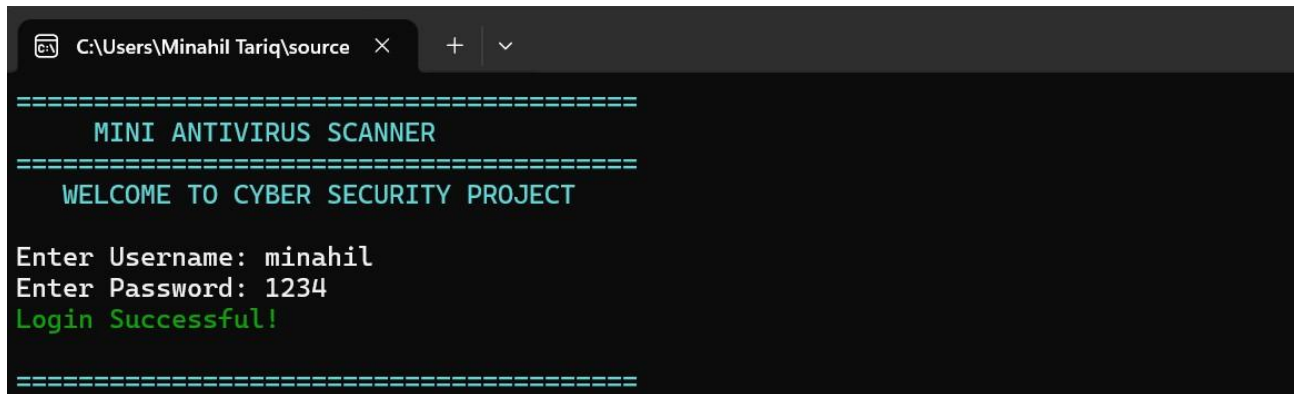
- App launches and displays LoginWindow (480 x 520 px, dark theme)
- On successful login (minahil / 1234), LoginWindow is destroyed and ScannerWindow is loaded (860 x 680 px)

- ScannerWindow has a left panel (scan buttons + results log) and a right panel (live report card + quarantine list)
- User selects a scan action; the scan_file() function is called for each file
- scan_file() performs three checks: filename keyword match, extension match, and file content byte read (first 2000 bytes)
- Results are written to the scrollable text box with colour-coded tags (green = safe, red = threat, yellow = details)
- Quarantine panel and report card update live after every file scanned

6. Features Assembly Backend

6.1 Secure Login System

The program opens with a cyan-coloured ASCII banner and prompts for a username and password. The LoginSystem procedure uses MyStrCompare to perform exact string matching. On failure a red error message is displayed and the user must try again there is no attempt limit, simulating a locked terminal. On success a green confirmation is shown before the menu loads.



```

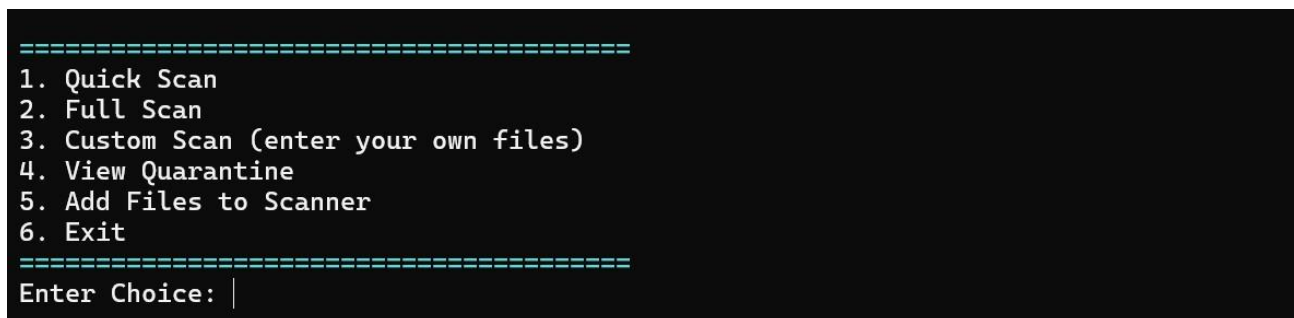
C:\Users\Minahil Tariq\source
=====
  MINI ANTIVIRUS SCANNER
=====
  WELCOME TO CYBER SECURITY PROJECT

Enter Username: minahil
Enter Password: 1234
Login Successful!
=====

```

6.2 Main Menu

A six-option menu is displayed after login. The user types a number and the program dispatches to the correct section using CMP / JE branching. Invalid input loops back to the menu.



```

=====
1. Quick Scan
2. Full Scan
3. Custom Scan (enter your own files)
4. View Quarantine
5. Add Files to Scanner
6. Exit
=====
Enter Choice: |

```

6.3 Quick Scan

Scans the first 4 built-in files. Uses a LOOP instruction with ECX as counter and ESI as a pointer into the files DWORD array. Each pointer is dereferenced with MOV EBX, [ESI] and passed to DetectThreat. Results appear immediately per file.

```
=====
Enter Choice: 1

Scanning Files...
safe.txt
--> SAFE FILE
virus.exe
--> THREAT DETECTED
Threat Level : LOW
project.doc
--> SAFE FILE
trojan.bat
--> THREAT DETECTED
Threat Level : MEDIUM

===== SCAN REPORT =====
Files Scanned : 4
Threats Found : 2
Safe Files    : 2
=====
```

6.4 Full Scan

Scans all files in the database — 8 built-in files plus any user-added extras. Empty extra slots are skipped by checking if the first byte at the pointer address is zero. A single ShowReport call summarises the full session.

```
C:\Users\Minahil Tariq\source X + v
Enter Choice: 2

Scanning Files...
safe.txt
--> SAFE FILE
virus.exe
--> THREAT DETECTED
Threat Level : LOW
project.doc
--> SAFE FILE
trojan.bat
--> THREAT DETECTED
Threat Level : MEDIUM
notes.pdf
--> SAFE FILE
hacktool.exe
--> THREAT DETECTED
Threat Level : HIGH
photo.jpg
--> SAFE FILE
malware.vbs
--> THREAT DETECTED
Threat Level : HIGH

===== SCAN REPORT =====
Files Scanned : 8
Threats Found : 4
Safe Files : 4

=====
```

6.5 Custom Scan

Prompts the user to enter between 1 and 5 filenames manually. Each buffer is cleared with a 50byte zero loop before ReadString is called. The user can test any arbitrary filename including ones not in the built-in database.

```
C:\Users\Minahil Tariq\source x + v
=====
Enter Choice: 3

How many files to scan? (max 5): 4
Enter filename: malware.enc
Enter filename: trojan.jpg
Enter filename: task1.pdf
Enter filename: hack.xls

Scanning Files...
malware.enc
--> THREAT DETECTED
Threat Level : LOW
trojan.jpg
--> THREAT DETECTED
Threat Level : MEDIUM
task1.pdf
--> SAFE FILE
hack.xls
--> THREAT DETECTED
Threat Level : HIGH

===== SCAN REPORT =====
Files Scanned : 4
Threats Found : 3
Safe Files : 1
=====
```

6.6 Add Files to Scanner

Up to 5 extra files can be added to the built-in database. Each added filename is immediately passed to DetectThreat so it is scanned on the spot and quarantined if a threat is detected. The file then persists in the database for future Full Scans.

```
=====
Enter Choice: 5

How many files to add? (max 5): 4
Enter filename to add: minahil.enc
--> SAFE FILE
File added to scanner!
Enter filename to add: trojan.epj
--> THREAT DETECTED
Threat Level : HIGH
File added to scanner!
Enter filename to add: virus.jpg
--> THREAT DETECTED
Threat Level : HIGH
File added to scanner!
Enter filename to add: eman.pdf
--> SAFE FILE
File added to scanner!
=====
```

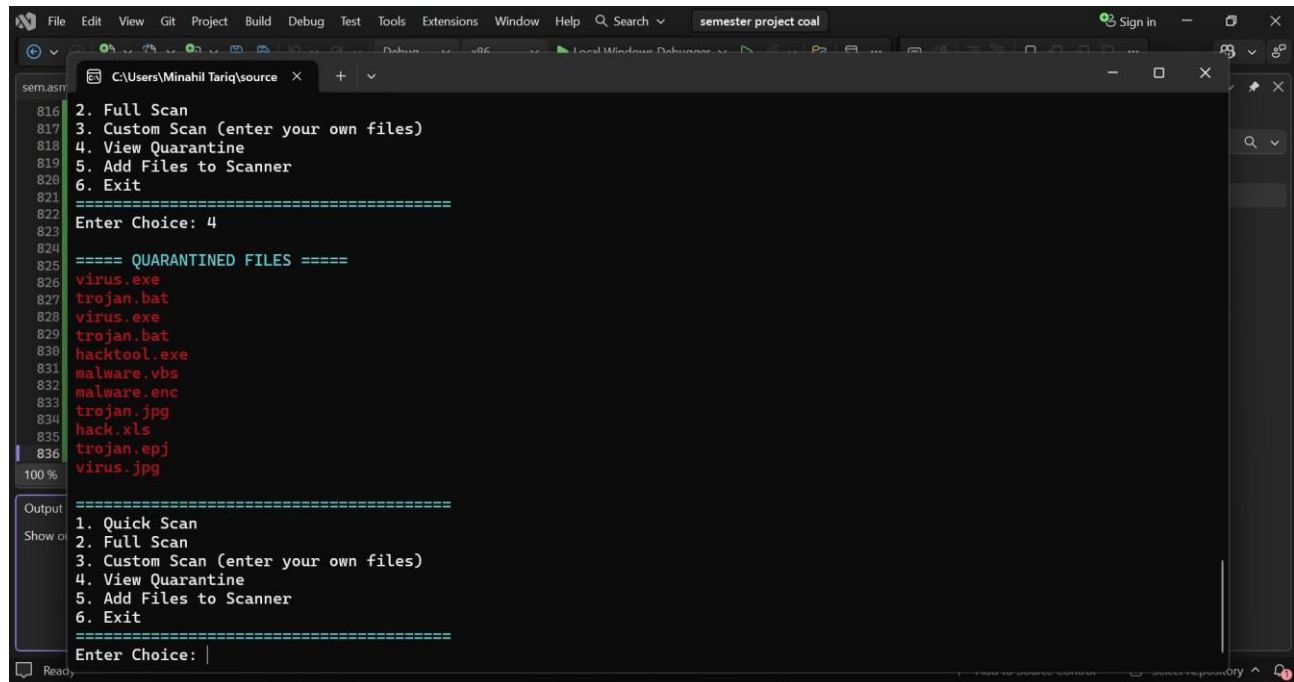
6.7 Threat Detection

DetectThreat calls ContainsWord four times, once per keyword (virus, trojan, malware, hack). On any match the filename is copied into the next free quarantine slot via MyCopy and the threat level

is shown based on the cumulative threatCount: 1 = LOW, 2 = MEDIUM, 3+ = HIGH. Safe files receive a green SAFE FILE message.

6.8 Quarantine & Persistent State

quarantineCount is a global DWORD that is never reset it grows across all scans in a single program run. The View Quarantine option lists all quarantined filenames in red. ResetCounters only zeroes the per-scan stats (filesScanned, threatCount, safeCount).



```
sem.asm
816 2. Full Scan
817 3. Custom Scan (enter your own files)
818 4. View Quarantine
819 5. Add Files to Scanner
820 6. Exit
821 =====
822 Enter Choice: 4
823
824 ===== QUARANTINED FILES =====
825 virus.exe
826 trojan.bat
827 virus.exe
828 trojan.bat
829 hacktool.exe
830 malware.vbs
831 malware.enc
832 trojan.jpg
833 hack.xls
834 trojan.epj
835 virus.jpg
836
100 %
Output
Show
1. Quick Scan
2. Full Scan
3. Custom Scan (enter your own files)
4. View Quarantine
5. Add Files to Scanner
6. Exit
Enter Choice: |
Ready
```

```
=====
Enter Choice: 4

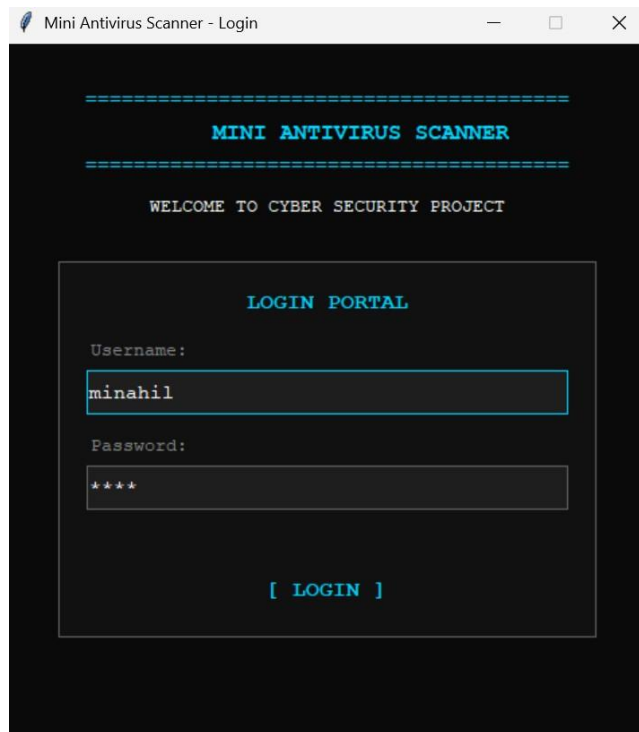
===== QUARANTINED FILES =====
virus.exe
trojan.bat
virus.exe
trojan.bat
hacktool.exe
malware.vbs
malware.enc
trojan.jpg
hack.xls
trojan.epj
virus.jpg
=====
```

7. Features Python GUI Frontend

7.1 Login Window

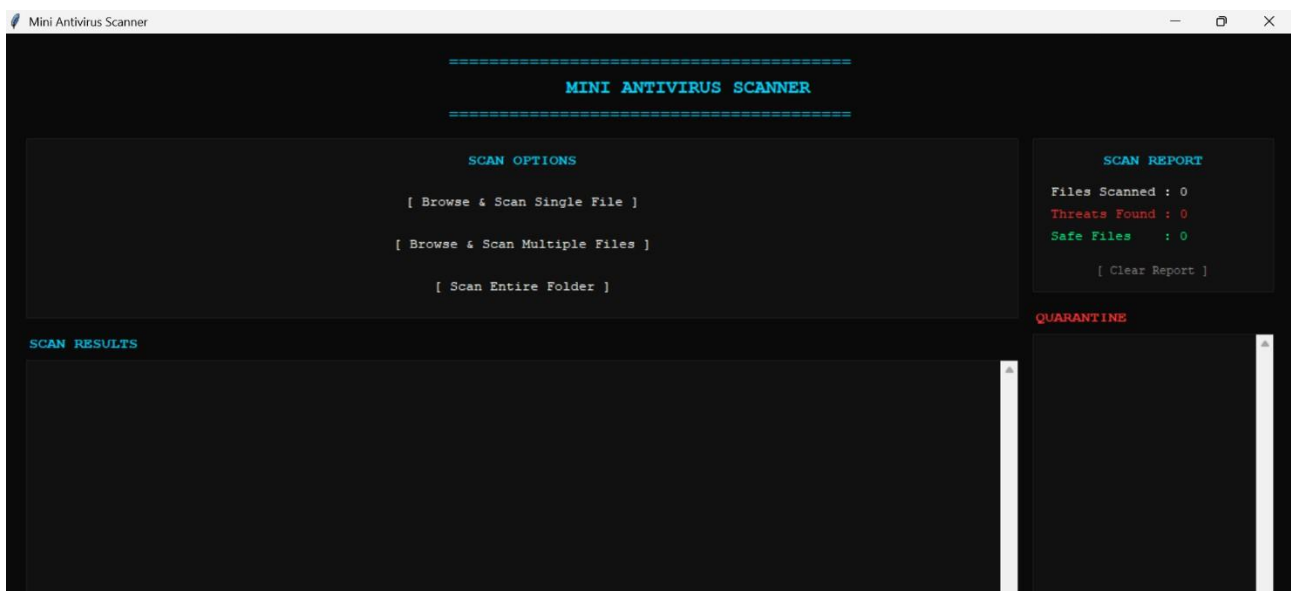
A 480 x 520 dark-themed window with a cyan-bordered login panel. The username and password fields use the Courier font consistent with the terminal aesthetic. The Enter key triggers the login

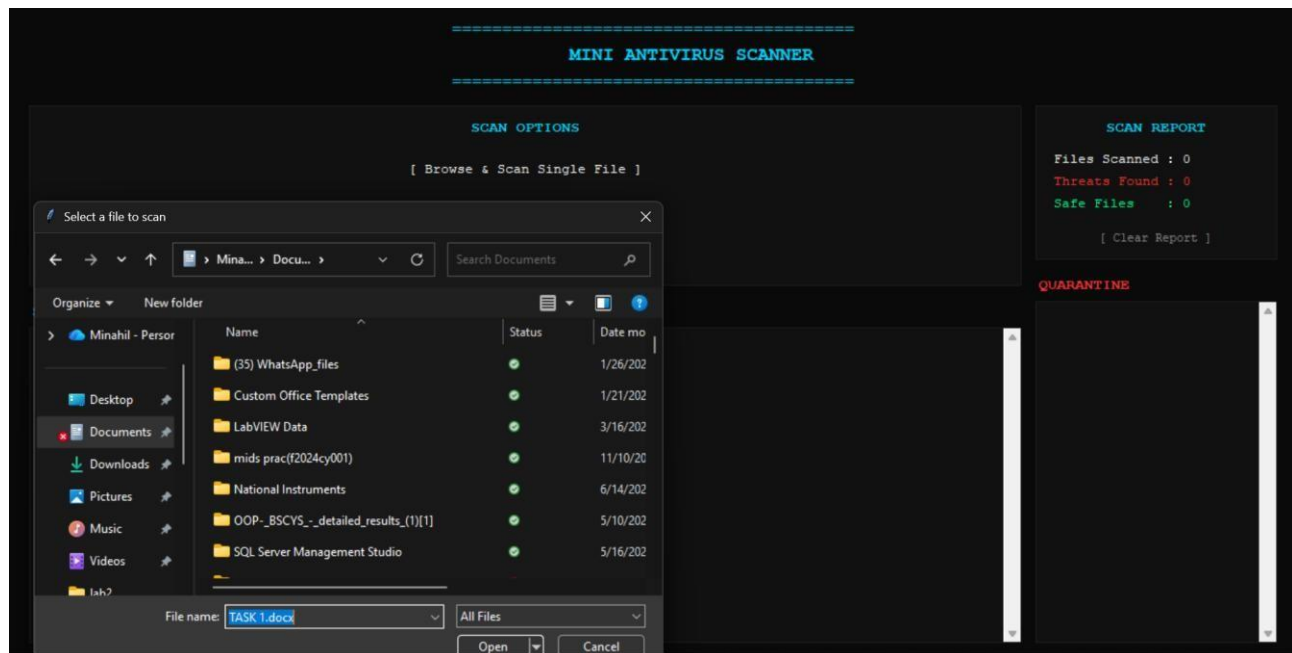
function. Wrong credentials display an attempt counter in red; correct credentials (minahil / 1234) show an 'Access Granted!' flash in green before the scanner window loads.



7.2 Browse & Scan Single File

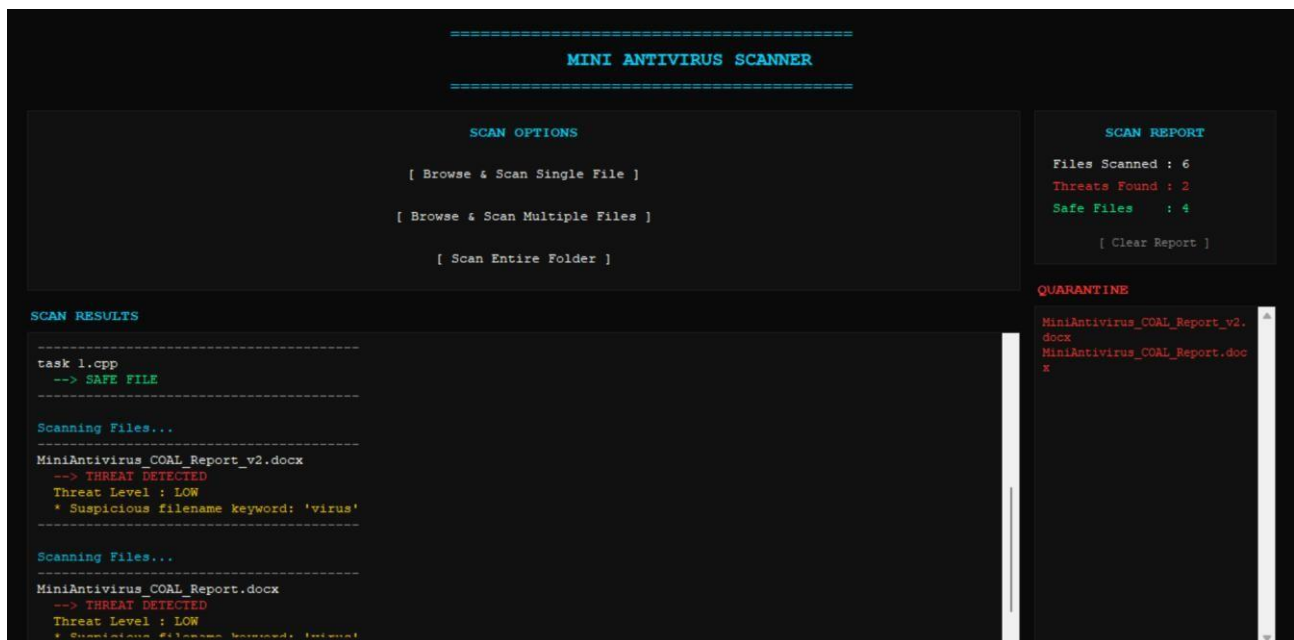
Opens an OS file-picker dialog filtered by common types (All Files, PDF, Word, Executables, Text Files). The chosen file is passed to `scan_file()` and results appear instantly in the results log.





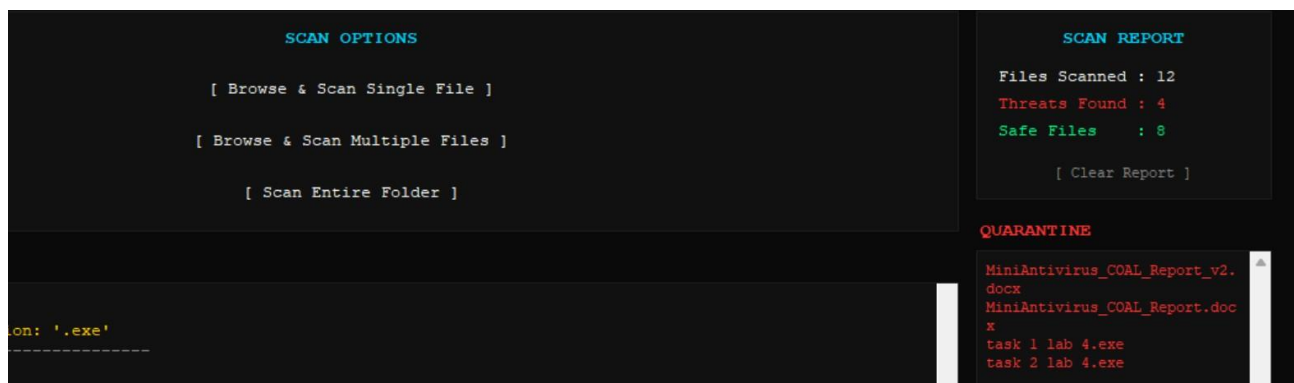
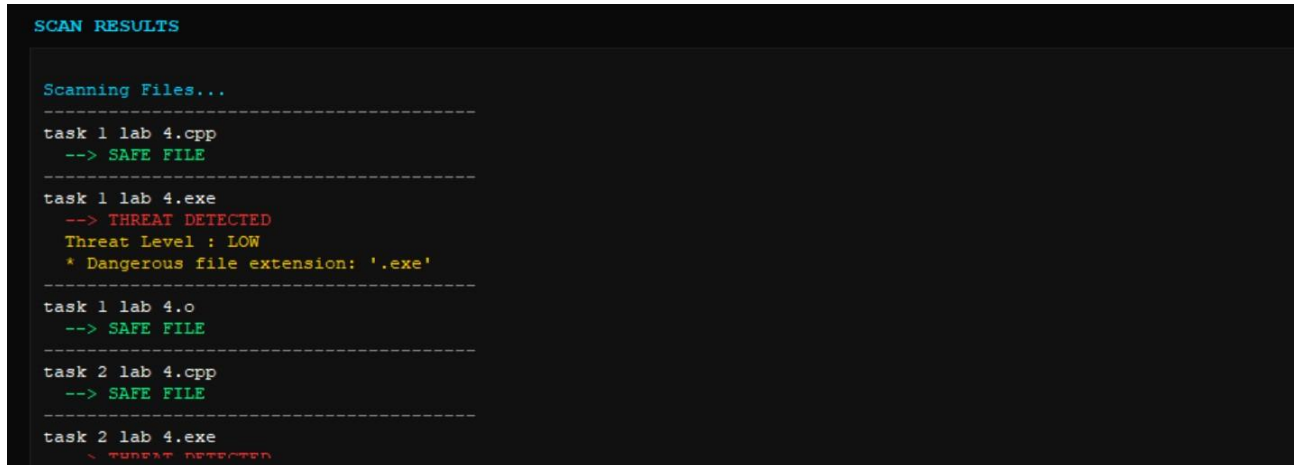
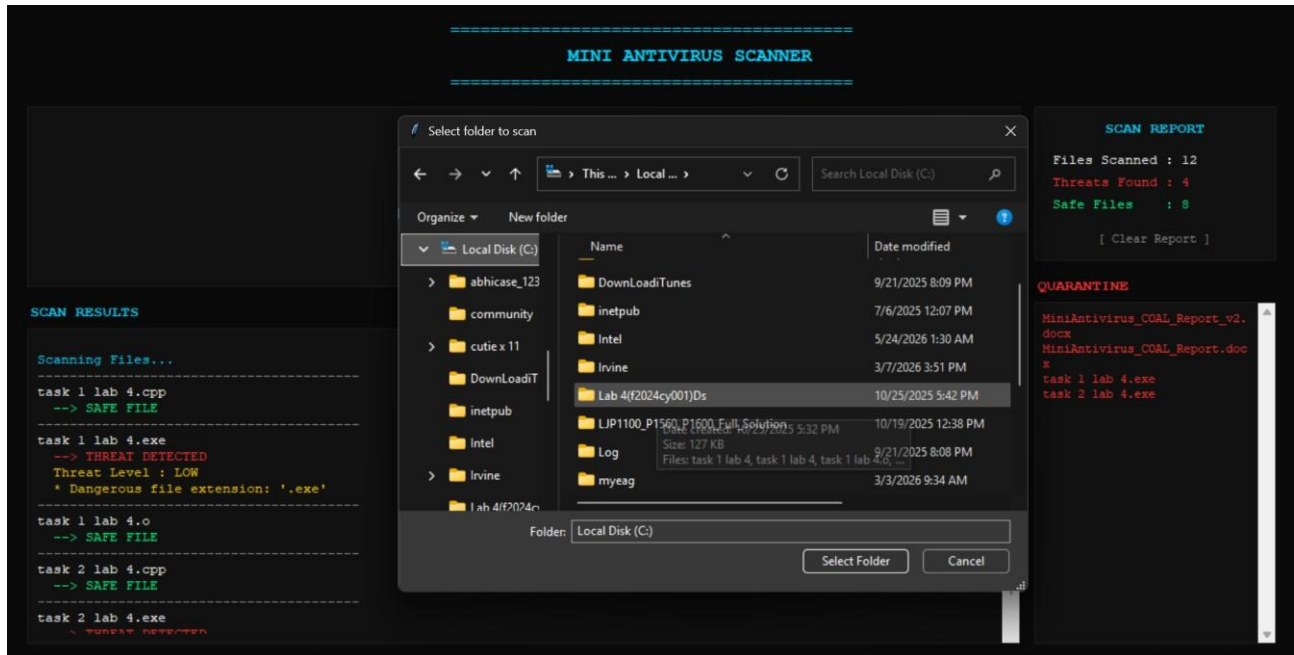
7.3 Browse & Scan Multiple Files

Opens a multi-select OS file picker (filedialog.askopenfilenames). All selected files are scanned in sequence. Each file gets its own result block with a separator line.



7.4 Scan Entire Folder

Opens a directory chooser. The application uses `os.listdir()` to enumerate all files in the selected folder (non-recursive, top-level only) and scans every file it finds. An info box appears if the folder is empty.



7.5 Three-Layer Detection Engine (scan_file)

The scan_file() function applies three independent checks to every file:

- Filename check: the basename (lowercased) is checked for 12 suspicious keywords such as virus, hack, malware, trojan, worm, spyware, keylog, exploit, payload, rootkit, backdoor, adware
- Extension check: the file extension is compared against a list of 6 dangerous extensions: .exe, .bat, .vbs, .cmd, .ps1, .scr
- Content check: the first 2000 bytes of the file are read as raw bytes, decoded to UTF-8 (errors ignored), and searched for 16 suspicious strings such as powershell, cmd.exe, shellcode, mimikatz, exploit, payload, ransomware, botnet, CreateRemote

Each matched item is added to a reasons list. The threat level is then: LOW (1 match), MEDIUM (2 matches), HIGH (3 or more matches).

7.6 Colour-Coded Results Log

The scrollable text area uses Tkinter text tags to colour output: green for SAFE FILE, red for THREAT DETECTED, yellow for threat level and reason details, cyan for section headers, white for filenames, and gray for separator lines.

7.7 Live Report Card

The right panel shows running totals Files Scanned, Threats Found, Safe Files updated in real time after each file. A 'Clear Report' button resets all counters, clears the log, and empties the quarantine list.

7.8 Quarantine Panel

Every threat filename detected is added to an in-memory list and displayed in a scrollable red text box on the right panel. The quarantine accumulates across multiple scans within the same session (cleared only by the 'Clear Report' button).

7.9 Status Bar

A bottom status bar updates after every scan action showing how many files were scanned, how many threats were found, and how many were safe. It also shows 'Ready' state and 'Scanning X file(s)...' during an active scan.

8. Code Explanation Assembly Core

8.1 String Comparison MyStrCompare

Performs exact matching between two null-terminated strings. ESI = string 1, EDI = string 2. Compares one byte at a time; if bytes differ, returns EAX = 0. If both reach null together, returns EAX = 1.

```
MyStrCompare PROC CmpLoop:    mov al, [esi]
; byte from string 1    mov bl, [edi]    ;
byte from string 2    cmp al, bl    jne CmpNo
; mismatch -> not equal    cmp al, 0    je
CmpYes    ; both null -> equal    inc
esi    inc edi    jmp CmpLoop CmpYes: mov eax,
1    ret
CmpNo:    mov eax, 0    ret
MyStrCompare ENDP
```

8.2 Substring Search ContainsWord

The heart of the detection engine. ESI = haystack (filename), EDI = needle (keyword). The outer loop moves along the haystack one character at a time. For each position the inner loop tries to match the full needle. If the inner loop reaches the needle's null terminator, the keyword was found.

```
ContainsWord PROC CW_Outer:    mov bl, [esi]    cmp bl,
0    ; end of haystack -> not found    je CW_No
mov ebx, esi    ; save current haystack position
mov ecx, edi    ; reset needle pointer
CW_Inner:    mov al, [ecx]    cmp al, 0    ;
end of needle = full match!
je CW_Yes    mov ah, [ebx]    cmp ah, 0    ;
haystack ended mid-match    je CW_Next    cmp al, ah
jne CW_Next    ; characters differ -> try next
inc ebx    inc ecx    jmp CW_Inner CW_Next:    inc esi
; advance haystack one position    jmp CW_Outer CW_Yes:
mov eax, 1    ret
CW_No:    mov eax, 0    ret
ContainsWord ENDP
```

8.3 Threat Detection DetectThreat

Receives filename address in EDX. Saves it to the global savedFileName DWORD, then calls ContainsWord for each of the four keywords. On any match it jumps to Found, copies the filename

into the quarantine slot via MyCopy, increments quarantineCount, and displays the threat level based on cumulative threatCount.

```
DetectThreat PROC
inc filesScanned
    mov savedFileName, edx    ; save because ContainsWord clobbers ESI

    mov esi, edx    mov edi,
OFFSET virusWord    call
ContainsWord        cmp eax, 1
je Found
    ; ... repeated for trojanWord, malwareWord, hackWord ...

    inc safeCount
    ; print green SAFE FILE message
ret

Found:    inc
threatCount
    ; print red THREAT DETECTED message
    ; copy filename to quarantine[quarantineCount] via MyCopy
inc quarantineCount    ; persistent - never reset
    ; print threat level LOW / MEDIUM / HIGH
ret
DetectThreat ENDP
```

8.4 Stack Discipline in Scan Loops

All scan loops push ECX and ESI before calling DetectThreat and pop them after. This preserves the loop counter (ECX used by LOOP) and the array pointer (ESI) which would otherwise be destroyed by internal Irvine32 calls.

```
QLoop:    push ecx                ; preserve loop counter
push esi    ; preserve array pointer    mov ebx,
[esi]    ; dereference -> get file address    mov edx,
ebx    call DetectThreat    ; may clobber ECX, ESI    pop
esi    ; restore    pop ecx    add esi, 4
; next DWORD pointer    loop QLoop    ; ECX--, jump
if != 0
```

11. Testing and Results

11.1 Assembly Backend Test Cases

Test Case	Input / Action	Expected Result	Status
Login Valid	minahil / 1234	Login Successful! (green)	PASS
Login Invalid	admin / wrong	Error message, retry loop	PASS

Quick Scan	Option 1	4 files; virus.exe + trojan.bat flagged	PASS
Full Scan	Option 2	8 files; 4 threats, 4 safe, HIGH level	PASS
Custom Scan	myvirus.txt	Keyword 'virus' matched, THREAT DETECTED	PASS
Custom Scan	report.pdf	No keyword match, SAFE FILE	PASS
Add Files	hacknew.exe	Immediately quarantined on add	PASS
Quarantine	Option 4	All threat filenames shown in red	PASS
Persistent Q	Multiple scans	quarantineCount grows, never resets	PASS
Threat Level	1 threat	LOW in yellow	PASS
Threat Level	3+ threats	HIGH in red	PASS

11.2 Python GUI Test Cases

Test Case	Input / Action	Expected Result	Status
GUI Login Valid	minahil / 1234	Green 'Access Granted!', scanner window loads	PASS
GUI Login Invalid	wrong creds	Red error with attempt count, no navigation	PASS
Scan .txt (clean)	Browse notes.txt	SAFE FILE in green	PASS
Filename keyword	Browse virus.txt	THREAT: 'virus' in filename	PASS

Ext detection	Browse file.vbs	.vbs flagged as dangerous extension	PASS
Content scan	File containing 'powershell'	THREAT: content keyword found	PASS
Multi-file scan	Select 5 files	All 5 scanned; report totals correct	PASS
Folder scan	Select folder	All top-level files scanned	PASS
Quarantine update	Scan threat file	Filename appears in red quarantine panel	PASS
Clear Report	Click Clear Report	All counters zero, log empty, quarantine empty	PASS
HIGH level	3+ matches in file	THREAT LEVEL: HIGH shown in yellow	PASS
Status bar	During scan	'Scanning X file(s)...' shown live	PASS

12. Challenges Faced

Assembly Backend

1. Register Preservation: ECX (loop counter) and ESI (array pointer) were being silently overwritten by Irvine32 I/O calls inside DetectThreat. Fixed by pushing ECX and ESI onto the stack before every DetectThreat call and popping them immediately after.
 2. savedFileName Global: DetectThreat calls ContainsWord four times. Each call modifies ESI, losing the original filename address. The solution was to save the incoming EDI to a dedicated DWORD variable (savedFileName) and reload ESI from it before each ContainsWord call.
 3. Pointer Arithmetic: The files array stores DWORD pointers (4 bytes each), so ESI advances by ADD ESI, 4. The raw extraFile buffers are consecutive 50-byte blocks, so their pointer advances by ADD ESI, 50 — a different stride that required careful management.
 4. Buffer Clearing: Reusing input buffers without zeroing them caused stale filenames from previous scans to pollute new scan results. Resolved with an explicit 50-byte zero-fill loop before every ReadString call.
 5. Persistent vs. Resettable State: Keeping quarantineCount growing across scans while resetting the per-scan stats required splitting the reset logic — ResetCounters touches only filesScanned, threatCount, and safeCount.
-

13. Conclusion

The Mini Antivirus Scanner successfully demonstrates cybersecurity principles at two levels of abstraction. The Assembly backend shows that meaningful threat detection logic — substring matching, secure login, quarantine management, and scan reporting — can be implemented entirely in low-level x86 machine instructions without any external libraries beyond the Irvine32 I/O helpers. The Python GUI frontend demonstrates how the same core security concepts translate into a practical, user-friendly desktop application that performs real file analysis.

Key takeaways from this project include: the importance of register discipline and stack management in Assembly; how modular procedure design maps directly to the concept of functions in high-level languages; how pattern-matching algorithms scale from simple filename checks in Assembly to multilayer content analysis in Python; and how a clean, dark-themed GUI can significantly improve the usability and presentation of a security tool.

Future enhancements could include: hash-based detection (MD5/SHA-1 signatures) in both layers; heuristic scoring based on file behaviour patterns; recursive folder scanning in the GUI; databasedriven threat definitions that can be updated without recompiling; and export of scan reports to PDF or CSV.
